Matching Engine & LOB

João Matheus Del Vecchio França Barbosa

Matching Engine & LOB - Planejamento

Como eu desenhei meu projeto, que ordem eu segui e quais foram meus planos.

I. **Requisitos** – Dado o enunciado, estabelecer e escrever quais os requisitos meus sistema deve satisfazer.
II. **Testes** – Pensar e escrever o conjunto de testes, ainda em alto nível, que eu vou precisar para garantir que todos estes requisitos sejam satisfeitos.
III. **Entidades e Interfaces** – Definir o paradigma, as interfaces do meu código e seus métodos e domínios. (A ideia era ter isso em mente antes para facilitar a construção dos testes)
IV. **Estrutura de Dados** – Escolher e pensar nas Estrutura de Dados que eu pretendo usar, a fim de satisfazer otimização temporal e de memória
V. **TDD** – Escolher a ordem de implementação dos testes e iniciar o ciclo TDD de: *Implementa Teste -> Coda o mínimo pro teste passar -> Refatore -> Repita...*

Todo esse rascunho de planejamento de requisitos inicial está catalogado em scratch/scratch_timeline.md

Matching Engine & LOB - Requisitos

Essa parte era bem simples, apenas traduzir do Enunciado o que era requisitado

Requisitos – Dado o enunciado, estabelecer e escrever quais os requisitos meus sistema deve satisfazer.

- *Como essa era uma etapa mais fácil, eu dei uma resumida em muitos requisitos, e omiti alguns que envolviam regras de negócio, o importante, no fim, era ter uma referência quando for elaborar meus testes, para facilitar a tarefa.*

➔ Quando receber uma ordem do tipo *Pegged*, *Limit* ou *Market*, deve ser capaz de processá-la segundo as regras de mercado correspondentes.
➔ Deve ser capaz de realizar operações de cancelamento e modificação de ordens ativas.
➔ Deve seguir uma regra de execução baseada primeiramente no preço e, em seguida, na ordem de chegada
➔ Quando uma ordem sofrer alteração de preço, deve ser capaz de colocá-la na faixa adequada, rebaixando sua prioridade de chegada na fila.
➔ Deve ser capaz de tratar e processar corretamente o *matching* de casos complexos e sequências de pedidos.
➔ Quando um *trade* for realizado, deve ser capaz de emitir e exibir os detalhes da execução imediatamente na saída do sistema.
➔ Deve ser capaz de exibir o estado atual do Limit Order Book para visualização.
➔ Deve ser capaz de suportar erros de entrada typo ou ignorar comandos não existentes.
➔ Deve realizar trade de ordens limits sempre que cruzarem o spread.
➔ Os algoritmos devem ser, quando possível, no máximo O(N)

Todo esse rascunho de planejamento e de requisitos inicial está catalogado em scratch/scratch_timeline.md

Matching Engine & LOB - Testes

Escolher escopo de Testes Unitários, em Linguagem Natural

Testes – Pensar e escrever o conjunto de testes, ainda em alto nível, que eu vou precisar para garantir que todos estes requisitos sejam satisfeitos.

- *A ideia era escrever os testes ainda em alto nível, mas também escrevendo o tipo de entrada que eu meu sistema precisa aceitar, com o output esperado, de tal forma a cobrir e garantir todos os comportamentos possíveis.*

```
7. Testa se consegue realizar uma ordem a mercado de compra quando existe uma limit de venda com liquidez maior que a ordem de compra

>>> limit sell 20 100
>>> market buy 40
Trade, price: 20, qty: 40

Ao printar o book:
Ordens de Compra
Ordens de Venda
60 @ 20

8. Testa se consegue realizar uma ordem a mercado de compra quando existe uma limite de venda com liquidez menor que a ordem de compra, mas existe outra ordem de
venda posterior com liquidez suficiente

>>> limit sell 20 30
>>> limit sell 25 100
>>> market buy 50
Trade, price: 20, qty: 30
Trade, price: 25, qty: 20

Ao printar o book:
Ordens de Compra
Ordens de Venda
80 @ 25
```

Todo esse rascunho de planejamento de testes inicial está catalogado em Scratches/scratch_tests.md

Matching Engine & LOB - Testes

Escolher escopo de Testes Unitários, em Linguagem Natural

- O Foco aqui é um conjunto de testes que valide o comportamento do código, **independente de como ele futuramente estruturado.**
- Os Testes passarem ou não deve **independer** da estrutura de dados e da lógica interna do código que decidirei mais na frente.
- Dessa forma, futuramente, **o código pode sofrer mudanças, sem precisar alterar a lógica de testes.**

No final dessa etapa, foram um total de 65 testes planejados, embora outros necessitaram ser futuramente adicionados, pois, **toda vez que um erro diferente surgia no código, era criado um teste para cobrir esse potencial Erro**

Todo esse rascunho de planejamento de testes inicial está catalogado em Scratches/scratch_tests.md

Matching Engine & LOB - Testes

Escolher escopo de Testes Unitários, em Linguagem Natural

A criação de muitos testes exige que as **regras de negócio** sejam previamente definidas e estabelecidas com clareza.

- Exemplo de regras seguidas:
  - Limit Orders que, assim que são criadas, cruzam o spread, são imediatamente efetuadas, e, caso sobre alguma coisa, é então colocado esse restante no book[1]
  - Modificar uma ordem para uma quantidade menor, mantém sua prioridade temporal.
  - Modificar uma ordem para uma quantidade maior e/ou um preço diferente, reduz ao máximo sua prioridade temporal
  - …

Todas as considerações de regra de negócio se encontram em: Project Documentation/LOB_rules.md

[1]**Referência Principal:** *"TRADES, QUOTES AND PRICES: Financial Markets Under the Microscope", JEAN-PHILIPPE BOUCHAUD, Capítulo 1 e 3*

Matching Engine & LOB - Entidades e Interfaces

Escolher e modelar a arquitetura do código, especialmente as suas interfaces e os métodos como estas conversam

Entidades e Interfaces – Definir o paradigma, as interfaces do meu código e seus métodos e domínios. (A ideia era ter isso em mente antes para facilitar a construção dos testes)

- Paradigma Orientado a Objeto - Ideia de elementos que possuem estado, entidades que se encapsulam e compartilham características semelhantes, etc.
- Escolher quem serão minhas entidades (minhas classes) e seus “métodos de conversa entre si”
    - Todas classes e métodos tendo **responsabilidade única**
    - Encapsulamento onde necessário (Comunicação entre classes feita apenas pelos métodos)
    - Tornar fácil fazer **melhorias futuras** no código, como adicionar novas ordens
    - Manter cada **domínio do código** separado em diferentes camadas
    - Escolha de interface ser i**ndependente da escolha da estrutura de dados** (mesma interface funciona para diferentes estruturas de dados e lógicas de algoritmo internas)
    - Escolha de entidade e interface também deve ser **independente das regras de negócio** que serão escolhidas

Matching Engine & LOB - Entidades e Interfaces

Escolher e modelar a arquitetura do código, especialmente as suas interfaces e os métodos como estas conversam

Como pensei...

- Responsável/Domínio por controlar as entradas e processar qual tipo de comando: ***Command Parser***
- Responsável por armazenar as ordens e cuidar dos dados: ***Limit Order Book***
- Responsável por traduzir as regras de negócio para cada comando: ***Matching Engine***
- Responsável por garantir a visualização do book: ***Book Printer*** *and* ***Volume Profile Printer***
- Entidade que abstrai uma ordem: ***Order***
  - Entidades Filhas com características próprias: ***Limit Order*** *and* ***Pegged Order*** *(que herdam Order)*

Limit Order Book

- ➔ *get_all_positions()*
- ➔ *get_order()*
- ➔ *get_best_bid()*
- ➔ *get_best_ask()*
- ➔ *get_best_limit_bid_price()*
- ➔ *get_best_limit_ask_price()*
- ➔ *get_best_resting_bid()*
- ➔ *get_best_resting_ask()*
- ➔ *add_pegged_order()*
- ➔ *add_limit_order()*
- ➔ *update_pegged_order()*

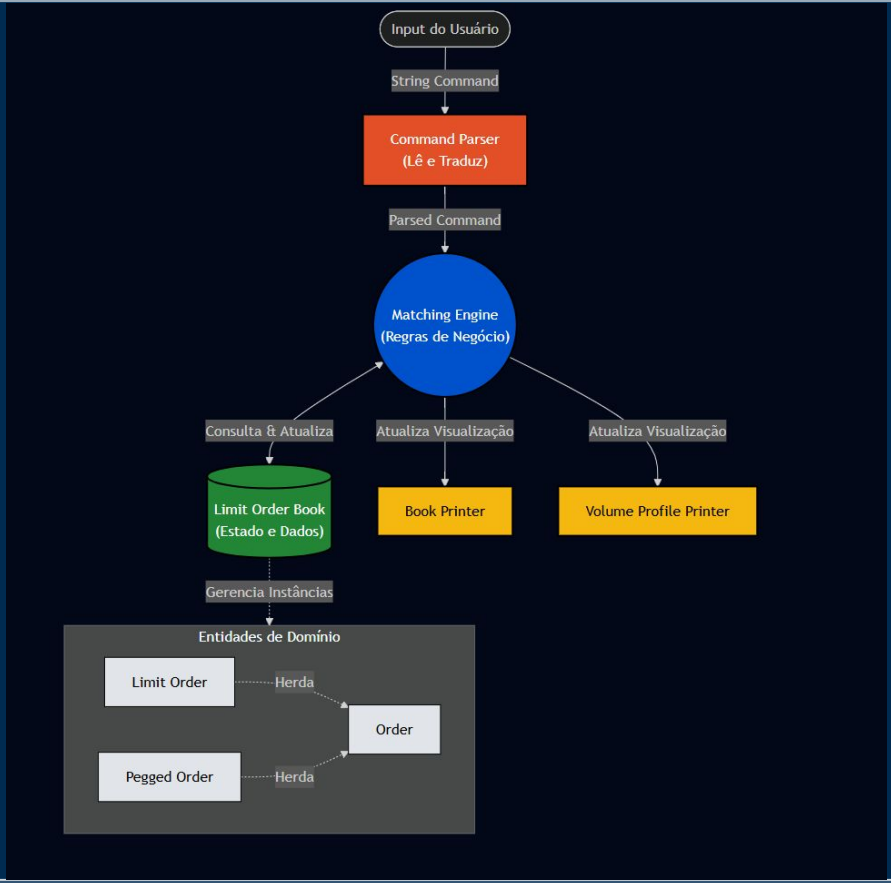
Matching Engine

- ➔ process_limit_order()
- ➔ process_market_order()
- ➔ process_pegged_order()
- ➔ process_modify_order()
- ➔ process_cancel_order()

→ *Defino como as entidades vão se comunicar, sem especificar como é a lógica e algoritmo interno delas*

Todas as entidades e seus métodos estão melhores descritos em Project Documentation/class_diagram.md

Matching Engine & LOB - Entidades e Interfaces



Todas as entidades e seus métodos estão melhores descritos em class_diagram.md

Matching Engine & LOB - Estrutura de Dados

Escolher a estrutura ótima em: Otimização de Tempo e Memória

- Ideia inicial de criar duas priority queues heapq, uma para as buy orders e outra para as sell orders
  - Inserção de limit em O(log n), e remover do topo também em O(log n). Para encontrar uma ordem no meio e remover, O(n).
- Hipótese: Inserção de Ordens Limites ocorrem em quantidades muito altas, logo existe uma necessidade de tornar esta inserção em ~ O(1) na maioria dos casos
- Solução: Uma mistura de 3 estruturas
  - Um Dicionário que mapeia orderID para um ponteiro de sua respectiva ordem
  - Dois Dicionários (buy and sell) que mapeiam um nível de preço existente a uma lista duplamente encadeada de ordens neste nível
  - Dois Arrays (buy and sell) ordenados para guardar os níveis de preço existente, tanto para buy quanto para bid

Matching Engine & LOB - Estrutura de Dados

Escolher a estrutura ótima em: Otimização de Tempo e Memória

Desse modo, e com a hipótese de que M, K, P << Número de ordens total

- Inserir Limit Order no Book
  - Melhor caso: O(1) – A ordem é passiva, não cruza o spread, e o preço especificado já existe no book. Inserir no dicionário e na lista encadeada são ambos O(1)
  - Caso mais frequente: de O(1) a O(M) – hipótese que na maior parte dos casos, a ordem é passiva e vai para um preço já existente. O(M) apenas se não existir o nível
  - Pior caso O(K + M*M + P*J_n) – Limit order que atravessa o spread e consome totalmente K ordens, remove todos M níveis, e cada pegged order gasta O(J) para localizar sua Prior. Temporal. – **Caso extremamente improvável**

L = num de limit orders no book, P = num pegged, M = num níveis, K = Número de ordens consumidas, J_n = num de ordens em um nível de preço

Matching Engine & LOB - Estrutura de Dados

Escolher a estrutura ótima em: Otimização de Tempo e Memória

Desse modo, e com a hipótese de que M, K, P << Número de ordens total

- Market Buy/Sell
  - Melhor caso: O(1) – A ordem é pequena e consome apenas a primeira ordem do livro
  - Caso mais frequente: O(K) – hipótese que na maior parte dos casos, a ordem apenas consome K ordens do topo, sem limpar todo o nível.
  - Pior caso O(K + M*M + P*J_n) – Varre **todos** níveis de preço, consome todas K ordens, remove todos níveis de preço do array bid prices em O(M) cada, e, ao alterar o novo limite, ativa o radar de peggeds em O(J) cada atualização de cada pegged order – **Caso extremamente improvável**

L = num de limit orders no book, P = num pegged, M = num níveis, K = Número de ordens consumidas, J_n = num de ordens em um nível de preço

Matching Engine & LOB - Estrutura de Dados

Escolher a estrutura ótima em: Otimização de Tempo e Memória

Desse modo, e com a hipótese de que M, K, P << Número de ordens total

- Inserção Peg Bid Buy/Offer Sell
  - O(1) sempre
- Cancel
  - Melhor Caso/Mais Recorrente: O(1) – A ordem não é a única que suportava o nível de referência das peggeds orders
  - Pior Caso: O(M + P*J_n) – A ordem cancelada é a única que suportava o nível de referêncIa das peggeds orders
- Modify
  - Melhor Caso O(1) – Apenas se reduz a quantidade
  - Caso mais recorrente: O(1) a O(M + P*J_n) – Modifica a qty/preço de tal modo que ainda não cruza spread.
  - Pior caso: O(K + M*M + P*J_n) – Altera-se o preço para um nível agressivo, que cruza spread, consumindo todos M níveis, e depois se fixa no book como novo melhor preço, atualizando as peggeds – **caso extremamente improvável**

L = num de limit orders no book, P = num pegged, M = num níveis, K = Número de ordens consumidas, J_n = num de ordens em um nível de preço

Matching Engine & LOB - TDD

Ciclo TDD

Escolher um teste → Criar o mínimo de código que faz o teste passar → Refatorar → Repita

De resto, é o código…

Apêndice

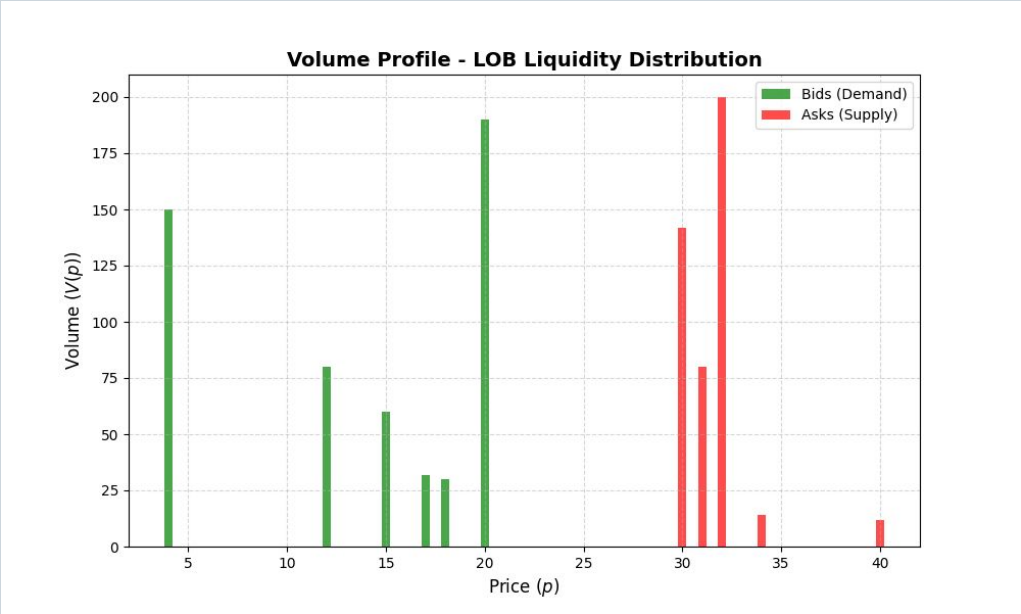
João Matheus Del Vecchio França Barbosa

Matching Engine & LOB - Apêndice

Volume Profile

Também implementei uma outra forma de olhar o book, visualizando o seu Perfil de Volumes

>>> print volume profile



Fim.

João Matheus Del Vecchio França Barbosa